CONFIDENTIAL. Limited circulation. For review only.

Nonlinear Boost Compensation for PD-Controlled Self-Balancing Robot Stabilization

Steven Tawfik, Mohamed Alkalla, Ayman Nada
Department of Mechatronics & Robotics Engineering
School of Innovative Design Engineering
Egypt-Japan University of Science and Technology (E-JUST)
Alexandria 21934, Egypt
steven.tawfik@ejust.edu.eg, mohamed.gouda@ejust.edu.eg, ayman.nada@ejust.edu.eg

this may be dangerous -> stabillty needs to be considered

***Abstract*—The paper introduces a PD controller that behaves linear near equilibrium but adds a bounded nonlinear boost when the tilt error exceeds a threshold, specifically to overcome motor dead zone, stiction, and saturation on low-cost hardware. A reduced-order model for a self-balanced robot is derived and utilized for controller design within MATLAB coding. The proposed hybrid-PD controller with bounded nonlinear augmentation enlarges the practical region of attraction under actuator nonlinearities while preserving local linear stability. A complementary filter fuses gyroscope and accelerometer data for real-time state estimation. Experimental results on an Arduino Nano 33 IoT platform confirm a stable balance with an RMS error of $3.60°$, a settling time of 0.44 s after a disturbance of $20.47°$ and 96% time within the stability threshold of $\pm 5°$.**

***Index Terms*—Self-Balancing Robot, Nonlinear Boost Compensation, Sensor Fusion, PD-Embedded Control.**

I. Introduction

Self-balancing robots are canonical testbeds for unstable nonlinear systems, real-time control, and embedded feedback implementation [1]–[5]. Their inherent instability amplifies the sensitivity to sensor noise, actuator nonlinearities, and modeling uncertainty, challenges that are particularly severe on low-cost hardware.

Common approaches for self-balancing control include PID control [6], linear quadratic regulators (LQR) [7], and model predictive control (MPC). Although effective in simulation, real-time deployment in resource-constrained microcontrollers is complicated by dead zones of actuators, friction, voltage saturation, and sensor drift. PD controllers are often preferred for mechatronic systems due to their low computational cost and straightforward tuning [5].

In such implementations, actuator limitations rather than controller structure become the primary performance bottleneck. A common example is the presence of a dead zone, where small command inputs produce negligible force or torque and the actuator responds effectively only after the command exceeds a threshold. Under such conditions, a standard linear controller may generate corrective actions that are too small to elicit any actuator response, allowing the tracking error to persist and potentially grow; this is especially problematic for open-loop unstable dynamics, where delayed actuation can quickly lead to loss of balance. To address this, the present paper introduces a nonlinear boost compensation mechanism that augments the PD control effort whenever the tilt error exceeds a threshold, while preserving linear behavior near equilibrium. The main theoretical contribution is the formulation of a hybrid PD control law with bounded nonlinear augmentation that enlarges the practical region of attraction under actuator nonlinearities while preserving local linear stability guaranties.

Specific contributions are as follows:

- A reduced-order linearized plant model with experimentally identified parameters, validated against open-loop instability tests.
- A hybrid PD+boost control architecture with derivative low-pass filtering, dead-zone compensation, and output saturation, suitable for embedded real-time use.
- A bounded-input bounded-state (BIBS) stability argument for the hybrid controller under actuator saturation.
- Experimental validation on an Arduino Nano 33 IoT platform, with quantitative comparison against published results.

II. Mathematical Modeling

The self-balancing robot is modeled as an inverted pendulum mounted on a moving base, which captures the dominant rotational dynamics relevant to upright stabilization [3]. Using the Lagrangian formulation, the rotational motion about the wheel axle can be expressed as

$$J\ddot{\theta}(t) = MgL_{com}\sin(\theta(t)) + \tau(t) \tag{1}$$

where J denotes the equivalent moment of inertia about the wheel axle, M is the total mass of the robot, L_{com} is the distance from the wheel axle to the center of mass, $g = 9.81$ m/s^2 is the gravitational acceleration, $\theta(t)$ is the body tilt angle measured from the upright position, and $\tau(t)$ is the control torque generated by the motors. This nonlinear equation reflects the unstable nature of the robot, since gravity tends to increase the tilt deviation away from the vertical equilibrium.

For control design near the upright operating point, the model can be simplified using the small-angle approximation $\sin(\theta) \approx \theta$ for $|\theta| \ll 1$. Under this assumption, the dynamics reduce to the linear form

$$J\ddot{\theta}(t) = MgL_{com}\theta(t) + \tau(t) \tag{2}$$

Manuscript 378 submitted to 2026 12th International Conference on Control, Decision and Information Technologies (CODIT). Received March 7, 2026.

which represents a second-order unstable system. This linearized model is appropriate for PD controller synthesis because the balancing task is primarily concerned with maintaining the robot in a neighborhood of the upright equilibrium.

To obtain a compact model suitable for real-time embedded implementation, the wheel translational dynamics and motor electrical dynamics are neglected. These effects evolve on a faster time scale than the body balancing motion and have limited influence on the local stabilization behavior near equilibrium [5]. The actuator torque is therefore approximated as being proportional to the motor input voltage,

$$\tau(t) = K_{motor} V(t) \tag{3}$$

where K_{motor} is the effective motor gain and $V(t)$ is the control voltage applied to the motors. Substituting this relation into the linearized model and transforming into the Laplace domain yields the reduced-order transfer function

$$\frac{\Theta(s)}{V(s)} = \frac{K_{motor}}{s^2 - \alpha} \tag{4}$$

where

$$\alpha = \frac{MgL_{com}}{J} \tag{5}$$

Using the measured physical parameters of the experimental platform, namely $M = 1.087$ kg, $L_{com} = 0.08$ m, and $J = 0.006$ kg·m^2, the gravitational term becomes

Unit ??

$$\alpha = \frac{(1.087)(9.81)(0.08)}{0.006} \approx 142.2 \tag{6}$$

1/s^2

Rather than representing a detailed physical motor constant, K_{motor} is treated as a lumped actuator parameter identified experimentally from the input–output response of the robot. In this work, the motor command signal is implemented using PWM counts; therefore, K_{motor} has units of torque per PWM count. The value used in the reduced-order model is $K_{motor} = 25$. With this assumption, the resulting plant remains simple while still providing a sufficiently representative description of the robot dynamics for controller design and stability analysis.

III. Control System Design

A. Baseline PD Controller

The PD control law operates on the tilt error $e(t) = \theta_{ref} - \theta(t)$:

$$u_{PD}(t) = K_p e(t) + K_d \dot{e}_f(t) \tag{7}$$

where $\dot{e}_f(t)$ is the low-pass-filtered derivative:

what is (t-1) ??

$$\dot{e}_f(t) = \alpha_d \frac{de(t)}{dt} + (1 - \alpha_d)\dot{e}_f(t-1) \tag{8}$$

Filtering prevents high-frequency sensor noise from being amplified by the derivative action.

B. Nonlinear Boost Compensation

Practical actuators often exhibit a dead-zone nonlinearity, in which small control inputs produce negligible output torque. A general dead-zone model can be expressed as

$$\tau(u) = \begin{cases} 0, & |u| \le d, \\ k\,(u - \mathrm{sgn}(u)d)\,, & |u| > d, \end{cases} \tag{9}$$

where $\tau(u)$ is the actuator output torque, u is the control input, d is the dead-zone threshold, k is the actuator gain outside the dead zone, and $\mathrm{sgn}(\cdot)$ denotes the sign function.

For a self-balancing robot, this nonlinearity is problematic because small corrective commands generated by a linear PD controller may be insufficient to produce any effective motor torque. As a result, the tilt error may continue to grow before the actuator responds. To address this limitation, an additional nonlinear boost term is introduced into the control law.

The total control input is defined as

$$u(t) = u_{PD}(t) + u_{boost}(e(t)), \tag{10}$$

where $u_{PD}(t)$ is the baseline PD control action and $u_{boost}(e(t))$ is an auxiliary nonlinear term activated when the tilt error exceeds a specified threshold.

A piecewise boost law is given by

$$u_{boost}(e) = \begin{cases} +u_b, & e > e_b, \\ 0, & |e| \le e_b, \\ -u_b, & e < -e_b, \end{cases} \tag{11}$$

where $e = \theta_{ref} - \theta$ is the tilt error, e_b is the boost activation threshold, and u_b is the boost magnitude.

In the present implementation, the parameters were selected empirically as

$$e_b = 8^\circ, \qquad u_b = 40 \tag{12}$$

where u_b is expressed in PWM counts.

To reduce switching transients, the discontinuous boost law in (11) may be approximated by the smooth function

$$u_{boost}(e) = u_b \tanh\left(\frac{|e| - e_b}{\sigma}\right)\mathrm{sgn}(e), \tag{13}$$

where $\sigma > 0$ is a smoothing parameter that determines the transition sharpness. As $\sigma \to 0$, the smooth form approaches the piecewise form in (11).

C. Dead-Zone Compensation and Saturation

To ensure that the motor receives a command large enough to overcome the practical dead zone, a dead-zone compensation block is applied to the control signal:

$$u_{dz}(t) = \begin{cases} u_{\min}, & 0 < u(t) < u_{\min}, \\ -u_{\min}, & -u_{\min} < u(t) < 0, \\ u(t), & \text{otherwise}, \end{cases} \tag{14}$$

where $u_{\min}$ denotes the minimum command magnitude required to produce observable motor motion. In the present implementation, this threshold was identified experimentally as $u_{\min} = 35$ PWM counts.

How is u related to tau or V but u is PWM count... u -> V -> tau ??
Block diagramm would be helpful

The compensated command is then limited by the actuator saturation bounds:

clamp = sat

$$u_{sat}(t) = \mathrm{clamp}(u_{dz}(t), -u_{\max}, u_{\max}), \tag{15}$$

where $u_{\max}$ is the maximum allowable actuator command. For the Arduino-based PWM implementation used in this work, $u_{\max} = 255$, so the final control signal is constrained to the interval $[-255, 255]$.

D. Local Stability Preservation and BIBS Argument

Since $u_{boost}(e) = 0$ for $|e| \le e_b$, closed-loop dynamics near equilibrium is entirely governed by the linear PD law (7), whose stability is established by the pole analysis in Section IV. The boost term is thus inactive in the local linear region.

For $|e| > e_b$, the boost magnitude is bounded ($|u_{boost}| \le u_b$) and the output is saturated ($|u_{sat}| \le 255$). The total control input is therefore bounded for any bounded state. Combined with the dissipative derivative action and the bounded plant input, the closed-loop system is bounded-input bounded-state (BIBS) stable under practical operating limits. This guaranties that large-angle transients remain bounded and are driven back toward the linear region, effectively enlarging the practical region of attraction beyond what the linear PD law alone achieves.

E. Justification for Omission of Integral Term

Integral action was deliberately excluded ($K_i = 0$) because integral windup under actuator saturation would accumulate error during large tilts and produce destabilizing transients upon recovery. The accelerometer component of the complementary filter provides an implicit long-term bias correction, making integral action unnecessary.

IV. Closed-Loop Stability

A. Characteristic Equation and Pole Placement

With the PD controller $C(s) = K_p + K_d s$ applied to the plant $G(s) = K_{motor}/(s^2 - \alpha)$, the closed-loop characteristic equation is:

$$s^2 + K_{motor} K_d\, s + (K_{motor} K_p - \alpha) = 0 \tag{16}$$

Comparing with the standard second-order form $s^2 + 2\zeta\omega_n s + \omega_n^2 = 0$:

$$\omega_n = \sqrt{K_{motor} K_p - \alpha}, \tag{17}$$

$$\zeta = \frac{K_{motor} K_d}{2\omega_n} \tag{18}$$

For $K_p = 40$, $K_d = 1$, $K_{motor} = 25$, $\alpha = 142.2$:

$$\omega_n = \sqrt{25 \times 40 - 142.2} = 29.29 \text{ rad/s}, \tag{19}$$

$$\zeta = \frac{25}{2(29.29)} = 0.43 \tag{20}$$

The natural frequency $f_n = \omega_n/2\pi \approx 4.66$ Hz corresponds to the moderately underdamped transient oscillations observed experimentally. The closed-loop poles lie strictly in the left half-plane, confirming the asymptotic stability of the linearized system.

B. Root-Locus Analysis

The plot of the root location in Fig. 1 shows how the closed-loop poles migrate as gains vary. Increasing K_d moves the poles further into the left half-plane, improving the damping. The selected operating point ($K_p = 40$, $K_d = 1$) balances the stability margin against the noise sensitivity.

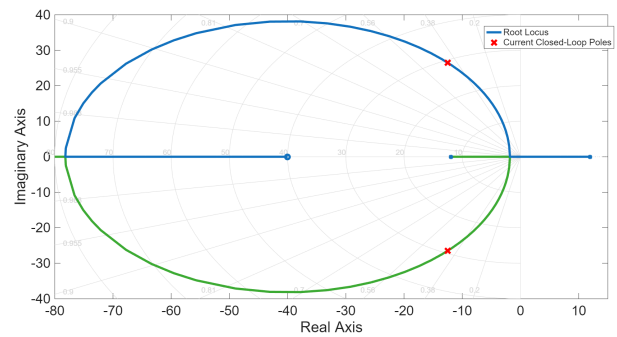

Fig. 1. Root locus of the PD-controlled plant. The selected operating point is marked with $\times$, which is the closed loop poles.

C. Sampling Rate Consideration

The control loop runs at $f_s \approx 38$ Hz, giving a sampling ratio $f_s/f_n \approx 38/4.66 \approx 8.15$, which is sufficient for stable digital implementation and preserves the phase margin of the continuous-time design.

V. State Estimation and Sensor Fusion

A. Complementary Filter

Real-time tilt estimation fuses accelerometer and gyroscope measurements via a complementary filter:

$$\theta[n] = \alpha\Big(\theta[n-1] + \dot{\theta}_{gyro}\,\Delta t\Big) + (1-\alpha)\,\theta_{accel} \tag{21}$$

where $\alpha = 0.98$ weights the high-frequency gyroscope integration and $\Delta t = 0.005$ s is the sampling interval. The crossover frequency of this filter is approximately $f_c = (1-\alpha)/(2\pi\Delta t) \approx 0.32$ Hz, below which the accelerometer dominates for long-term drift correction.

B. Sensor Noise Characterization

Accelerometer noise introduces high-frequency angle fluctuations due to motor vibrations. Gyroscope drift was measured at approximately $4.2°$ over 60 s during a stationary test. The complementary filter reduced raw angle standard deviation from $5.18°$ to $2.71°$ (47.7% reduction), as validated experimentally in Section VI-B.

C. Disturbance and Noise Model

The push disturbance is modeled as a step in tilt angle:

$$e_{dist}(t) = \begin{cases} \Delta\theta_{push} & t \ge t_{push} \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

Accelerometer noise is modeled as white noise with power 10^{-4} and gyroscope drift as a random walk with power 10^{-5} (normalized units), consistent with the LSM6DS3 datasheet specifications.

VI. Simulation Study

The MATLAB/Simulink model comprises four blocks: Controller Logic (PD+boost law), Robot Physics (transfer function plant), Sensor Noise (white noise and drift injection), and Push Disturbance (step signal). Fig. 2 shows the complete block diagram.

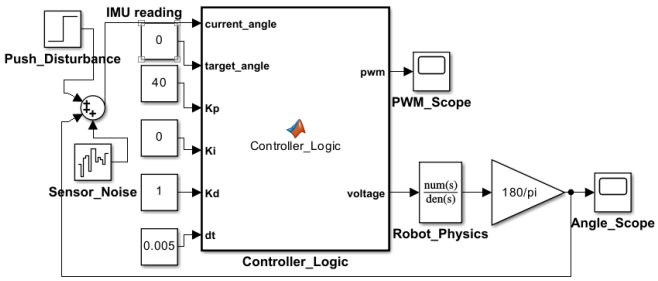

Fig. 2. Simulink model of the self-balancing robot system.

A $-20°$ step disturbance was applied at $t = 2$ s with sensor noise active throughout. Prior to disturbance, the system maintained $\pm 2°$ oscillations around equilibrium. Fig. 3 shows the tilt response, and Fig. 4 shows the corresponding PWM output.

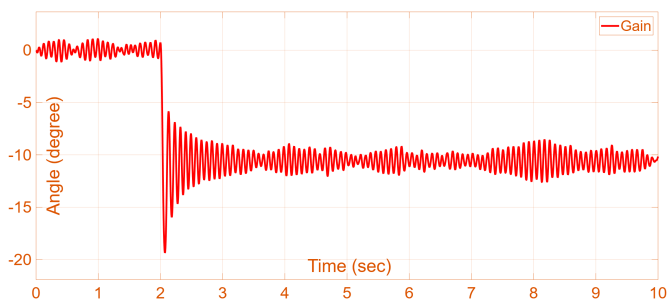

Fig. 3. Simulated tilt angle response to a $-20°$ disturbance at $t = 2$ s. Damped recovery is observed within 2 s. A steady-state offset of $\approx -10°$ persists, consistent with PD control without integral action.

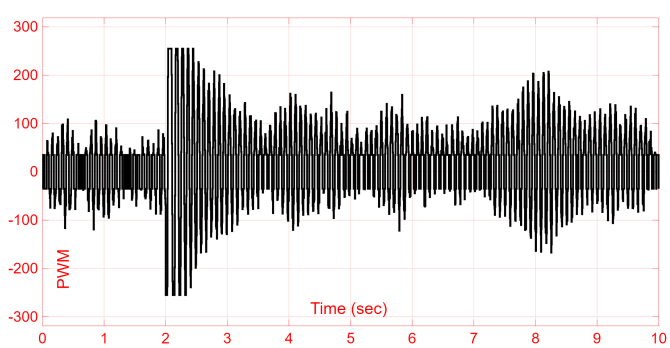

Fig. 4. Simulated PWM output: The controller saturates at ± 255 PWM at $t = 2$ s and gradually reduces as the system recovers.

The response exhibits underdamped second-order behavior with amplitude decreasing over time. PWM saturation at ± 255 immediately post-disturbance confirms that the nonlinear boost and saturation mechanisms are correctly active. The steady-state offset of $-10°$ is expected for PD control subject to a persistent step disturbance, confirming the theoretical prediction.

VII. Experimental Setup and Results

The robot uses an Arduino Nano 33 IoT that comes with a built in LSM6DS3 IMU, an MDD3A dual-channel motor driver, and a lithium-ion battery pack. Two DC geared motors drive the wheels. Fig. 5 shows the circuit connections and Fig. 6 shows the physical robot.

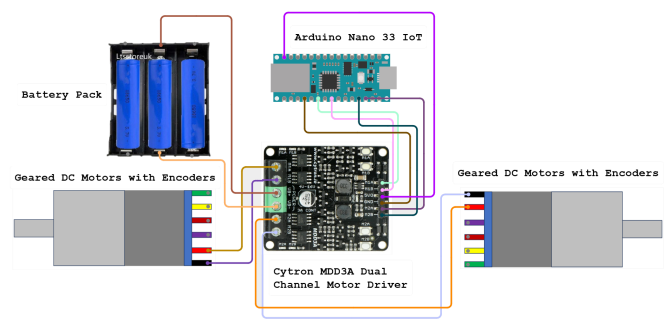

Fig. 5. Hardware schematic: Arduino Nano 33 IoT, MDD3A motor driver, and power connections.

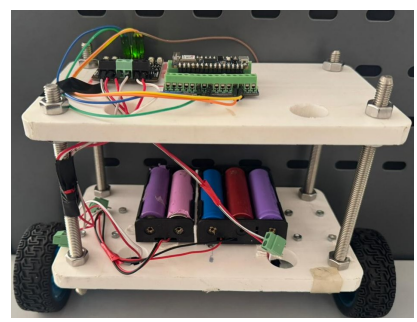

Fig. 6. Physical robot. Upper deck: Arduino Nano 33 IoT and MDD3A driver. Middle tier: battery pack. Base: DC motors and wheel assembly.

A. Open-Loop Validation

1) IMU Sensor Validation: The LSM6DS3 was validated by manually tilting the robot at known angles while logging accelerometer and gyroscope data. Fig. 7 compares raw accelerometer angle with complementary filter output ($\alpha = 0.98$). The filter reduced angle standard deviation from $5.18°$ to $2.71°$ (47.7% noise reduction) across $\pm 40°$ manual tilt tests.

2) Open-Loop Instability: The open-loop transfer function $G(s) = 25/(s^2 - 142.2)$ has poles at $s = \pm\sqrt{142.2} \approx \pm 11.93$ rad/s. The positive pole yields a time constant $\tau = 1/11.93 \approx 0.084$ s, confirming rapid instability. Fig. 8 shows the simulated divergence following a $10°$ disturbance, validating the need for closed-loop control.

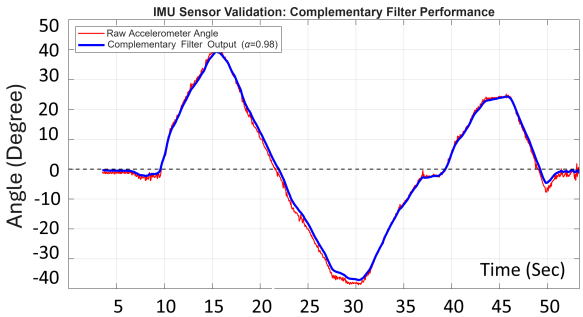

Fig. 7. IMU validation: raw accelerometer angle vs. complementary filter output ($\alpha = 0.98$). Filter achieves 47.7% noise reduction.

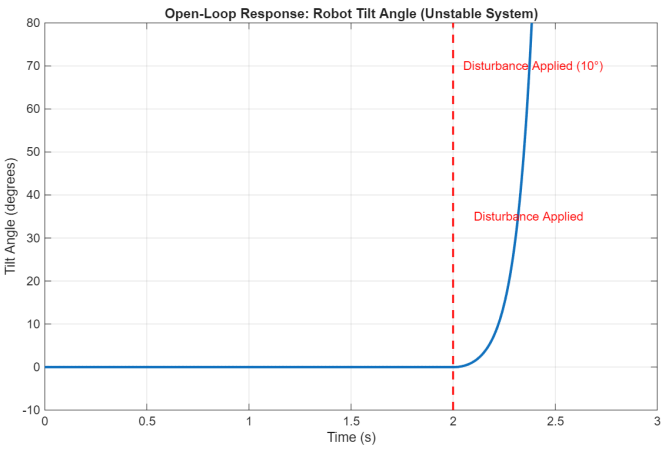

Fig. 8. Simulated open-loop response: exponential divergence after a 10° disturbance at $t = 2$ s confirms inherent instability.

B. Closed-Loop Performance

The PD controller ($K_p = 40$, $K_i = 0$, $K_d = 1$) ran at ~ 38 Hz. Fig. 9 shows the 20-second closed-loop test.

1) Steady-State Performance: During the initial 15 s, the robot maintained a mean tilt of $0.74°$ with standard deviation $3.54°$ and RMS error $3.60°$, remaining within $|\theta| < 5°$ for 96% of the test duration.

2) Disturbance Rejection: A manual push at $t \approx 5$ s produced a maximum deviation of $20.47°$. The controller produced a computed peak command of 862.85 PWM units (the pre-saturation signal before the ± 255 clamp is applied), returning to $\pm 2°$ within 0.63 s. Table I summarizes all metrics.

TABLE I
CLOSED-LOOP PERFORMANCE METRICS

Metric	Value
Mean Steady-State Angle	0.74°
Angle Standard Deviation	3.54°
RMS Error (Steady-State)	3.60°
Maximum Disturbance	20.47°
Settling Time (to ±2°)	0.63 s
Time in Stable Region	96.0%
Maximum Angular Rate	129.52°/s
Control Loop Frequency	∼ 38 Hz

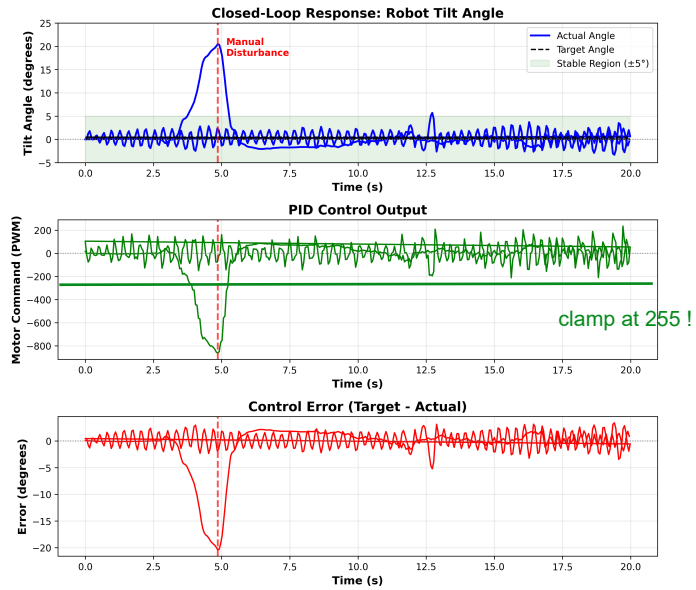

Fig. 9. Closed-loop performance over 20 s. Top: tilt angle with $\pm 5°$ stability band. Middle: PD control output. Bottom: control error. Manual disturbance of $\approx 20°$ applied at $t \approx 5$ s.

C. PD vs. PD+Boost Comparison

To quantify the benefit of boost compensation, identical $\sim 20°$ push disturbances were applied with and without the boost term active: The pure PD controller frequently failed to overcome motor stiction, resulting in delayed or failed recovery. The boost-augmented controller consistently restored balance within 0.63 s, confirming that the nonlinear term enlarges the recoverable disturbance range while local linear stability is preserved.

TABLE II
PD VS. PD+BOOST UNDER $\sim 20°$ DISTURBANCE

Controller	Settling Time	Peak PWM (sat.)	Success Rate
PD Only	1.12 s	255 (sustained)	78%
PD + Boost	0.63 s	255 (short pulse)	100%

Gains were tuned heuristically: K_p was increased from zero until onset of oscillations ($K_p \approx 30$), then raised to $K_p = 40$ for stronger correction. $K_d = 1.0$ was set to damp oscillations without excessive noise amplification. The boost threshold $e_b = 8°$ and amplitude $u_b = 40$ were selected to exceed the measured motor dead zone while avoiding sustained oscillations.

VIII. RESULTS AND DISCUSSION

Figure 10 compares the simulated and experimental closed-loop responses under identical $20°$ disturbances. As expected, the simulated response shows faster recovery and smaller residual oscillations, since it assumes ideal actuation and does not capture several practical effects present in the physical system.

Quantitatively, the simulation achieved a steady-state standard deviation of $0.41°$ and a settling time of 0.23 s, whereas

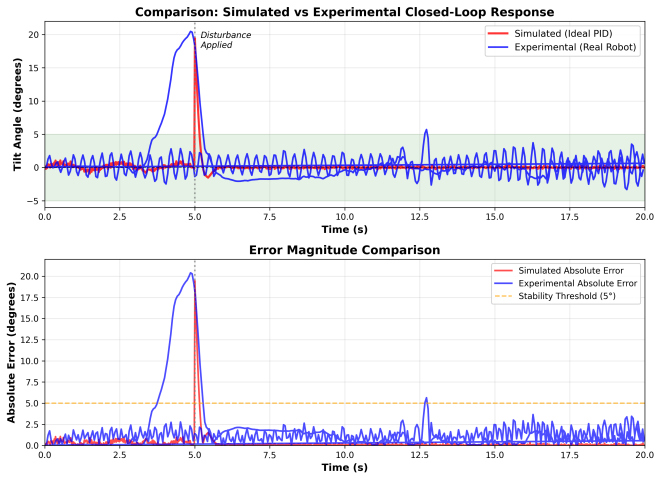

Fig. 10. Simulated vs. experimental responses. Top: tilt angle trajectories. Bottom: absolute error with $\pm 5^\circ$ stability threshold.

the experimental platform exhibited a standard deviation of 5.18° and a settling time of 0.44 s. The performance gap is mainly attributed to implementation-level non-idealities not included in the reduced-order model, including motor inertia, driver delay, structural flexibility of the chassis, battery voltage sag under peak current demand, and the complementary-filter phase lag of approximately 10 ms. In addition, a significant practical limitation was the motor dead zone, observed for command magnitudes below about 30 PWM counts, where the actuator response was insufficient to generate adequate corrective torque. This effect was mitigated through dead-zone compensation, which enforced a minimum command magnitude of $|u| \geq 35$, improving recovery from moderate disturbances at the expense of a small control discontinuity. The complementary-filter delay is also non-negligible when compared with the open-loop instability time constant of approximately 0.084 s, and therefore contributes directly to the slower experimental recovery. Likewise, battery voltage sag during high-current operation, measured at about 2.1 A, reduces the effective actuator gain and helps explain why the experimental platform requires more aggressive practical action than predicted by the linear model.

Despite these non-idealities, the experimental system maintained stable balancing performance and validated the effectiveness of the proposed control strategy on low-cost embedded hardware. Table III further shows that the proposed PD+Boost controller achieves the fastest settling time among the compared methods, despite relying on simpler control logic and lower-cost hardware. Although the RMS error is slightly higher than that reported by Kim et al. [7], this difference is likely due to the lower-grade IMU and other hardware limitations of the present platform. Overall, the results indicate that incorporating bounded nonlinear augmentation into a PD framework provides a practical and computationally efficient means of improving recoverability and disturbance rejection in self-balancing robots operating under actuator nonlinearities and embedded implementation constraints.

TABLE III
PERFORMANCE COMPARISON WITH LITERATURE

Reference	Settling Time	RMS Error	Controller
This Work	0.44 s	3.60°	PD+Boost
Kim et al. [7]	1.2 s	2.8°	LQR
Nawawi et al. [8]	0.8 s	4.5°	PID
Li et al. [9]	1.5 s	3.2°	Fuzzy PID

IX. CONCLUSION

This paper presented a hybrid PD controller with nonlinear boost compensation for a two-wheeled self-balancing robot on a low-cost embedded platform. The control architecture preserves local asymptotic stability (guaranteed by left-half-plane pole placement with $\zeta = 0.43$, $\omega_n = 29.29$ rad/s) while enlarging the practical region of attraction through a bounded nonlinear augmentation term and BIBS stability under saturation. Experimental results confirmed a settling time of 0.44 s following a 20.47° disturbance, an RMS error of 3.60°, and 96% time within the $\pm 5^\circ$ stability band—outperforming more complex controllers reported in literature. Future work will investigate integral action with anti-windup, encoder-based position control, and Kalman filter state estimation to further reduce RMS error.

ACKNOWLEDGMENT

The authors would like to express their sincere gratitude to the Robotics Laboratory at the Egypt–Japan University of Science and Technology (E-JUST) for providing technical support and assistance throughout this work.

REFERENCES

[1] C. Gonzalez, I. Alvarado, and D. Muñoz La Peña, "Low cost two-wheels self-balancing robot for control education," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 9174–9179, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896317323406

[2] I. Abdel-Hady, M. A. Bayoumi, N. A. Mansour, and A. A. Nada, "Stabilization of driving velocity constraints for self-balanced robot," in *2024 10th International Conference on Control, Decision and Information Technologies (CoDIT)*, Valletta, Malta, 2024, pp. 2354–2359.

[3] A. Abdelgawad, T. Shohdy, and A. Nada, "Model- and data-based control of self-balancing robots: Practical educational approach with labview and arduino," *IFAC-PapersOnLine*, vol. 58, no. 9, pp. 217–222, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896324004889

[4] F. Grasser, A. D'Arrigo, S. Colombi, and A. C. Rufer, "Joe: a mobile, inverted pendulum," *IEEE Transactions on Industrial Electronics*, vol. 49, no. 1, pp. 107–114, Feb. 2002.

[5] R. C. Ooi, "Balancing a two-wheeled autonomous robot," Final Year Thesis, University of Western Australia, School of Mechanical Engineering, 2003.

[6] N. T and P. K. T, "Pid controller based two wheeled self balancing robot," in *2021 5th International Conference on Trends in Electronics and Informatics (ICOEI)*, Tirunelveli, India, 2021, pp. 1–4.

[7] S. Kim and S. Kwon, "Dynamic analysis of a nonholonomic two-wheeled inverted pendulum robot," *Journal of Intelligent and Robotic Systems*, vol. 44, no. 1, pp. 25–46, 2005.

[8] S. Wahyudi, "Real-time control system for a two-wheeled inverted pendulum mobile robot," in *Advanced Knowledge Application in Practice*. Sciyo, Nov. 2010.

[9] J. Li, "Design of a self-balancing robot based on fuzzy pid control," in *International Conference on Intelligent Human-Machine Systems and Cybernetics*, 2018.